CampusMatch Architecture

CampusMatch — Backend Architecture (Supabase)
1. Auth Design
2. Schema (Core Tables)
2.1 `universities`
2.2 `profiles`
2.3 `posts`
2.4 `stories`
2.5 `story_views`
2.6 `likes` (post likes)
2.7 `profile_likes` ("Like Profile" button on user profile)
2.8 `threads` (DM conversation)
2.9 `messages`
2.10 `matches`
2.11 `surprise_sessions`
2.12 `surprise_saves` ("Saved from Surprise")
2.13 `surprise_queue` (presence/matching)
2.14 `blocks`
2.15 `reports`
2.16 `notifications`
2.17 `events` (lightweight analytics)
3. Row Level Security (RLS) — Core Policies
3.1 Block enforcement on reads
4. Storage Buckets
5. Edge Functions
6. Realtime Channels
7. Matching Algorithm (Surprise)
8. Recommendation Logic ("Suggested for you" / "People you might match with")
9. Data Retention / Cleanup
10. Environment / Keys

CampusMatch — Backend Architecture (Supabase)

Stack: Supabase (Postgres + RLS, Auth, Storage, Realtime, Edge Functions/Deno).

---

1. Auth Design

- **Method:** Email OTP only — no passwords. Matches "verified student" requirement.
- **Flow:**
  1. Client submits university email → Edge Function `validate-university-email` checks domain against `universities` table.
  2. If valid, Supabase Auth `signInWithOtp` triggers, email sent via Resend (configured as Supabase's custom SMTP provider).
  3. On OTP confirm, a Postgres trigger creates a row in `profiles` linked to `auth.users.id`, with `verified = true` and `university_id` set.
  4. Guests never call `signInWithOtp` — no `auth.users` row exists, `auth.uid()` is null, RLS blocks all writes automatically.

---

2. Schema (Core Tables)

2.1 `universities`

Column	Type	Notes
id	uuid PK	
name	text	e.g. "University of Nairobi"
email_domain	text unique	e.g. "uon.ac.ke", used for OTP gating
badge_label	text	short label shown on cards, e.g. "UoN"

2.2 `profiles`

Column	Type	Notes
id	uuid PK, references `auth.users.id`	
username	text unique	
display_name	text	

avatar_url	text	Storage path
bio	text	
university_id	uuid FK → universities	
verified	boolean default false	
age	int	18+ enforced at sign-up
gender / interested_in	text/enum	optional, per product needs
interests	text[]	chips shown on profile
created_at	timestamptz	
is_banned	boolean default false	

2.3 `posts`

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → profiles	
type	enum('photo','video','gallery')	
media	jsonb	array of `{ url, type, order }` — supports gallery
caption	text nullable	max 100 chars enforced client + check constraint
audience	enum('everyone','matches','university')	
created_at	timestamptz	
deleted_at	timestamptz nullable	soft delete for "manage/delete posts"

2.4 `stories`

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → profiles	
media_url	text	
type	enum('photo','video')	
created_at	timestamptz	
expires_at	timestamptz	default `created_at + interval '24 hours'`

2.5 `story_views`

Column	Type	Notes
story_id	uuid FK → stories	
viewer_id	uuid FK → profiles	
viewed_at	timestamptz	composite PK (story_id, viewer_id)

2.6 `likes` (post likes)

Column	Type	Notes
post_id	uuid FK → posts	
user_id	uuid FK → profiles	
created_at	timestamptz	composite PK (post_id, user_id)

2.7 `profile_likes` ("Like Profile" button on user profile)

Column	Type	Notes
liker_id	uuid FK → profiles	
liked_id	uuid FK → profiles	
created_at	timestamptz	composite PK (liker_id, liked_id)

2.8 `threads` (DM conversation)

Column	Type	Notes

id	uuid PK	
user_a	uuid FK → profiles	lower id by convention
user_b	uuid FK → profiles	
status	enum('pending','matched')	pending = frozen until reply
initiator_id	uuid FK → profiles	who sent the first DM
message_count	int default 0	enforced ≤5 while status='pending'
created_at / updated_at	timestamptz	
unique (user_a, user_b)	constraint	one thread per pair

2.9 `messages`

Column	Type	Notes
id	uuid PK	
thread_id	uuid FK → threads	
sender_id	uuid FK → profiles	
content	text	
created_at	timestamptz	
read_at	timestamptz nullable	

2.10 `matches`

A thread transitions from `pending` → `matched` once the recipient replies. A **trigger** on `messages` insert checks: if the sender is not the thread's `initiator_id` and thread status is `pending`, set `threads.status = 'matched'` and insert a row into `matches` for both directions (or just flip the thread status and treat `matches` as a view over `threads where status='matched'` — simpler, recommended).

> Recommendation: skip a separate `matches` table; expose a Postgres **view** `matches_view` selecting `threads where status = 'matched'`, joined with both profiles. Less duplicated state to keep in sync.

2.11 `surprise_sessions`

Column	Type	Notes
id	uuid PK	
user_a / user_b	uuid FK → profiles	
started_at / ended_at	timestamptz	
signaling_channel	text	realtime channel name used for this call

2.12 `surprise_saves` ("Saved from Surprise")

Column	Type	Notes
user_id	uuid FK → profiles	who saved
saved_user_id	uuid FK → profiles	who was saved
session_id	uuid FK → surprise_sessions	
created_at	timestamptz	composite PK (user_id, saved_user_id)

2.13 `surprise_queue` (presence/matching)

Column	Type	Notes
user_id	uuid PK FK → profiles	
status	enum('waiting','in_call')	
joined_at	timestamptz	used for FIFO pairing

2.14 `blocks`

Column	Type	Notes
blocker_id	uuid FK → profiles	
blocked_id	uuid FK → profiles	
created_at	timestamptz	composite PK (blocker_id, blocked_id)

2.15 `reports`

Column	Type	Notes
id	uuid PK	
reporter_id	uuid FK → profiles	
reported_user_id	uuid FK → profiles	
context_type	enum('post','profile','surprise_session','message')	
context_id	uuid nullable	
reason	text	
created_at	timestamptz	
status	enum('open','reviewed','actioned') default 'open'	

2.16 `notifications`

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → profiles	recipient
type	enum('new_match','new_message','story_view','like',...)	
payload	jsonb	
read_at	timestamptz nullable	
created_at	timestamptz	

2.17 `events` (lightweight analytics)

Column	Type	Notes
id	uuid PK	
user_id	uuid nullable	null for guest events
event_name	text	e.g. "guest_blocked_action", "signup_completed"
metadata	jsonb	
created_at	timestamptz	

---

3. Row Level Security (RLS) — Core Policies

RLS is enabled on every table. General pattern:

```
-- profiles: anyone (incl. anon) can read non-banned profiles for Discover
create policy "public read profiles"
on profiles for select
using (is_banned = false);

-- profiles: only the owner can update their own row
create policy "own profile update"
on profiles for update
using (auth.uid() = id);

-- posts: readable by everyone if audience='everyone', restricted otherwise
create policy "read posts by audience"
on posts for select
using (
  deleted_at is null and (
    audience = 'everyone'
    or (audience = 'university' and exists (
      select 1 from profiles p
      where p.id = auth.uid() and p.university_id = (
        select university_id from profiles where id = posts.user_id
      )
    ))
    or (audience = 'matches' and exists (
      select 1 from threads t
      where t.status = 'matched'
      and ((t.user_a = auth.uid() and t.user_b = posts.user_id)
        or (t.user_b = auth.uid() and t.user_a = posts.user_id))
    ))
  )
```

```
);

-- posts: only owner can insert/update/delete
create policy "own posts write"
on posts for all
using (auth.uid() = user_id)
with check (auth.uid() = user_id);

-- likes: only authenticated, non-blocked users can like
create policy "insert own like"
on likes for insert
with check (auth.uid() = user_id);

-- messages: only thread participants can read/write, and only if not blocked
create policy "thread participants only"
on messages for select
using (
  exists (
    select 1 from threads t
    where t.id = messages.thread_id
    and (t.user_a = auth.uid() or t.user_b = auth.uid())
  )
);

create policy "send message if not blocked and within limit"
on messages for insert
with check (
  auth.uid() = sender_id
  and exists (
    select 1 from threads t
    where t.id = thread_id
    and (t.user_a = auth.uid() or t.user_b = auth.uid())
    and not exists (
      select 1 from blocks b
      where (b.blocker_id = t.user_a and b.blocked_id = t.user_b)
         or (b.blocker_id = t.user_b and b.blocked_id = t.user_a)
    )
  )
);
```

> The "≤5 messages while pending" rule and the block-list exclusion on feed/profile reads are enforced via a **combination of RLS + a Postgres trigger/function** (`check_message_limit()` below), since RLS alone can't easily count existing rows mid-insert without a `before insert` trigger.

```
 create or replace function check_message_limit()
returns trigger as $$
declare
  thread_status text;
  current_count int;
  initiator uuid;
begin
  select status, message_count, initiator_id into thread_status, current_count, initiator
  from threads where id = new.thread_id;

  if thread_status = 'pending' and new.sender_id = initiator and current_count >= 5 then
    raise exception 'Message limit reached until recipient replies';
  end if;

  -- if recipient is replying, flip thread to matched
  if thread_status = 'pending' and new.sender_id <> initiator then
    update threads set status = 'matched' where id = new.thread_id;
  end if;

  update threads set message_count = message_count + 1, updated_at = now()
  where id = new.thread_id;

  return new;
end;
$$ language plpgsql security definer;

create trigger trg_check_message_limit
before insert on messages
for each row execute function check_message_limit();
```

3.1 Block enforcement on reads

Every feed/profile/discover read policy should additionally exclude rows where a block exists in either direction. In practice this is centralized in a SQL function `not_blocked(other_user_id uuid)` reused across policies:

```
 create or replace function not_blocked(other_user_id uuid)
returns boolean as $$
  select not exists (
    select 1 from blocks
```

```
      where (blocker_id = auth.uid() and blocked_id = other_user_id)
         or (blocker_id = other_user_id and blocked_id = auth.uid())
    );
  $$ language sql stable security definer;
```

Used as `and not_blocked(posts.user_id)` appended to the relevant `using` clauses.

---

4. Storage Buckets

Bucket	Access	Contents
`avatars`	public read, owner write	profile pictures
`posts`	public read (subject to audience logic enforced at query layer, not storage), owner write	photo/video/gallery post media
`stories`	public read, owner write, lifecycle-cleaned after 24h	story media
`surprise-temp`	private, no persistence needed (WebRTC is P2P, not stored)	only used if a future "recording opt-in" feature is added — currently unused, no recording per spec

> Since `posts.audience` controls visibility logic at the database/query level (who *sees* a post in the feed), but Storage URLs are public, technically anyone with a direct URL could view restricted media. For matches-only/university-only posts, switch those buckets (or those specific objects) to **private** with **signed URLs** generated server-side (Edge Function) only after the RLS-backed `posts` query confirms the requester is allowed to see that post. Recommended for `matches` / `university` audience posts; `everyone` posts can stay public for CDN simplicity.

---

5. Edge Functions

Function	Trigger	Purpose
`validate-university-email`	called from sign-up form	checks domain against `universities`, blocks disposable/non-edu domains
`match-surprise-queue`	called when user taps Surprise / polled or realtime-triggered	pairs two `waiting` users in `surprise_queue` FIFO-style, creates `surprise_sessions` row, returns signaling channel name to both clients
`generate-signed-media-url`	called when rendering a `matches` / `university` audience post	issues a short-lived signed URL for private storage objects
`send-push-notification`	Postgres webhook on `notifications` insert	dispatches Web Push/FCM to the recipient's registered device(s)
`cleanup-expired-stories`	scheduled (cron)	deletes `stories` rows + storage objects past `expires_at`
`auto-ban-on-reports`	scheduled or webhook on `reports` insert	if `reports` against a user exceed threshold in a time window, sets `profiles.is_banned = true`

---

6. Realtime Channels

Channel	Purpose
`thread:{thread_id}`	new message events, read receipts
`presence:surprise`	who's currently in the Surprise queue (Supabase Presence)
`call:{session_id}`	ephemeral WebRTC offer/answer/ICE candidate exchange, torn down when call ends
`notifications:{user_id}`	live notification badge updates
`story:{user_id}`	live "new story posted" indicator on the stories bar

---

7. Matching Algorithm (Surprise)

```
1. User taps Surprise → upsert into surprise_queue (status='waiting', joined_at=now()).
2. Edge Function `match-surprise-queue` (invoked on insert via DB webhook, or polled
   client-side every couple seconds as fallback):
     - Selects the oldest two 'waiting' users (excluding blocked pairs, excluding the
       requester themselves) using `select ... for update skip locked` to avoid race
       conditions when multiple invocations run concurrently.
```

```
     - Creates a `surprise_sessions` row, sets both users' queue status to 'in_call'.
     - Publishes a realtime event on `presence:surprise` (or a dedicated per-user channel)
       with the session id + signaling channel name.
3. Both clients receive the pairing event, join `call:{session_id}`, perform WebRTC
   offer/answer/ICE handshake, connect P2P.
4. "Next" → client tears down peer connection, deletes its surprise_sessions end time,
   re-upserts into surprise_queue (status='waiting') to restart matching.
5. "End" → both clients leave call:{session_id}; session row gets ended_at; client shows
   Save Profile prompt → optional insert into surprise_saves.
```

---

8. Recommendation Logic (“Suggested for you” / “People you might match with”)

Simple v1 (no ML, pure SQL), runnable as a view or Edge Function:

```sql
create or replace view suggested_matches as
select p.id as suggested_user_id, requester.id as requester_id
from profiles p
cross join profiles requester
where p.id <> requester.id
  and p.is_banned = false
  and not_blocked(p.id)
  and p.university_id = requester.university_id  -- same-university weighting
  and not exists (  -- not already matched/messaged
    select 1 from threads t
    where (t.user_a = requester.id and t.user_b = p.id)
       or (t.user_b = requester.id and t.user_a = p.id)
  )
order by cardinality(p.interests & requester.interests) desc, random()
limit 20;
```

This can later be swapped for a proper recommendation Edge Function (collaborative filtering, embedding similarity, etc.) without changing the frontend contract — it just calls the same `suggested_matches` endpoint.

---

9. Data Retention / Cleanup

Data	Retention
Stories	24h, cleaned via `cleanup-expired-stories` cron
Surprise WebRTC media	never stored (P2P only, no recording)
Deleted posts	soft-deleted (`deleted_at`), hard-purge after 30 days via cron
Reports	retained indefinitely for moderation history (`report-history` screen)

---

10. Environment / Keys

- Client uses the **anon public key** only, relying entirely on RLS — never the `service_role` key in the browser.
- `service_role` key used only inside Edge Functions (server-side), e.g. for `auto-ban-on-reports` , `cleanup-expired-stories` , signed URL generation.
- Resend API key stored as a Supabase Auth SMTP secret, not exposed to client.